

XML (eXtensible Markup Language)

❖ Les principales applications de XML

XML est utilisé dans un large éventail d'applications qui nécessitent une structure de données flexible et interopérable :

- **Échange de données entre systèmes** : XML est couramment utilisé pour échanger des informations entre des applications et des serveurs, souvent dans le contexte des services web. Par exemple, des protocoles comme SOAP (Simple Object Access Protocol) se basent sur XML pour la structuration de données lors de la communication entre applications.
- **Formats de documents** : XML est à la base de nombreux formats de documents, tels que DocBook pour les manuels et les livres, ou encore XHTML pour les pages web structurant le contenu de manière standardisée.
- **Stockage de données** : XML est également utilisé pour enregistrer des configurations de logiciels ou des préférences utilisateur, notamment dans des fichiers de configuration (.xml) lisibles et éditables facilement.
- **Applications web et mobiles** : De nombreuses applications utilisent XML pour structurer les informations qu'elles récupèrent depuis un serveur. Par exemple, une application mobile qui reçoit des informations météo d'un serveur utilise souvent XML pour structurer les données météorologiques.

❖ Les bases du XML

Le XML, ou eXtensible Markup Language, est un langage informatique de balisage générique, principalement utilisé pour structurer, organiser et échanger des données de manière lisible par les humains et par les machines. En informatique, il existe plusieurs centaines de langages, qui peuvent être regroupés en trois grandes catégories :

- **Langages de programmation** : utilisés pour créer des logiciels et des applications, ils incluent des langages comme C, Java, C++, Objective-C, C#, PHP, Basic, Python, etc.
- **Langages de requête** : conçus pour interroger et manipuler des données dans des bases de données, comme SQL et XQuery.
- **Langages de description** : dédiés à la structuration et à la présentation des données, comme SGML, XML, ou HTML.

Les balises dans XML permettent de structurer les données de manière hiérarchique et organisée, ce qui facilite leur interprétation et leur exploitation par les systèmes informatiques.

❖ Les balises et les attributs

En XML, les balises sont des éléments essentiels qui délimitent et organisent les données. On distingue deux types de balises :

- **Balises par paires** : Ce sont les balises les plus courantes, qui encadrent le contenu avec une balise ouvrante et une balise fermante. Par exemple :

```
<titre>Introduction à XML</titre>
```

- **Balises uniques** : Elles ne contiennent pas de contenu et sont auto-fermantes. Par exemple :

```
<image source="logo.png" />
```

❖ Règles de nommage des balises

Pour garantir la validité et l'interopérabilité des documents XML, certaines règles de nommage des balises doivent être respectées :

- Les noms de balises peuvent contenir des lettres, des chiffres ou des caractères spéciaux.
- Ils ne doivent pas commencer par un nombre ni par un signe de ponctuation.
- Les noms de balises ne peuvent pas commencer par les lettres "XML" (quelle que soit la casse).
- Les noms ne doivent pas contenir d'espaces.
- Il est recommandé d'éviter certains caractères spéciaux tels que -, ;, ., <, et >, car ils peuvent être mal interprétés par les parsers XML.

❖ Les attributs dans XML

Les attributs sont utilisés pour ajouter des informations supplémentaires aux éléments, sans que ces informations ne soient le contenu principal. Les attributs apparaissent dans la balise ouvrante et suivent le format nom="valeur".

Exemple de balise et d'attribut XML :

```
<prix devise="FCFA">25000</prix>
```

Dans cet exemple, l'élément <prix> contient la valeur principale 25000, tandis que l'attribut devise fournit une information supplémentaire en précisant l'unité monétaire (FCFA).

❖ Syntaxe des commentaires

Les commentaires en XML permettent d'ajouter des notes ou des explications pour les lecteurs du code sans que ces informations ne soient interprétées par les programmes. La syntaxe d'un commentaire XML est la suivante :

```
<!-- Ceci est un commentaire -->
```

❖ Structure d'un document XML

Un document XML se compose de deux parties principales : le **prologue** et le **corps**.

- **Le prologue** : Il est situé en début de document et contient des informations importantes, telles que la version XML utilisée, le jeu de caractères et d'autres métadonnées. Il permet de définir les propriétés essentielles du document.

Par exemple :

```
<?xml version="1.0" encoding="UTF-8" standalone="yes" ?>
```

- **Le corps** : Il est constitué de l'ensemble des balises qui décrivent les données. Dans un document XML, une balise *racine* (ou élément racine) englobe tous les autres éléments. Cette balise racine est essentielle et doit contenir l'intégralité du contenu du document. Voici un exemple de structure avec une balise racine :

```
<racine>  
  <balise_paire>texte</balise_paire>  
  <balise_paire2>texte</balise_paire2>  
</racine>
```

Exemple de description d'un répertoire téléphonique :

```
<repertoire>  
  <personne>Bernard</personne>  
  <personne>Patrick</personne>  
</repertoire>
```

❖ Règles de formation d'un document XML valide

Pour qu'un document XML soit **bien formé**, il doit respecter certaines règles fondamentales :

1. **Prologue correctement renseigné** : Si le document utilise la version 1.1 de XML, le prologue doit être spécifié comme suit.
2. **Une seule balise racine** : Un document XML ne peut avoir qu'une seule balise racine qui englobe tous les autres éléments.
3. **Respect des règles de nommage** : Les noms des balises et des attributs doivent suivre les conventions de nommage définies pour XML.
4. **Balises en paires correctement fermées** : Chaque balise ouvrante doit avoir une balise fermante correspondante.
5. **Valeurs d'attributs entre guillemets** : Toutes les valeurs des attributs doivent être placées entre guillemets simples ou doubles.
6. **Pas de chevauchement des balises** : Les balises ne doivent pas se chevaucher. Le document doit suivre une structure hiérarchique claire et logique.

Exemple d'un document XML valide :

```
<?xml version="1.0" encoding="UTF-8" standalone="yes" ?>
<repertoire>
  <personne>Roger</personne>
  <personne>John</personne>
</repertoire>
```

Ce document respecte toutes les règles mentionnées et peut être validé à l'aide d'un éditeur XML tel qu'Editix, qui vérifiera sa conformité.

Exercice : Création d'un document XML simple avec balises et attributs

Objectif : Apprendre à créer un document XML structuré avec des balises et des attributs, en respectant les règles de syntaxe et de nommage.

Consignes :

1. Créez un document XML représentant une bibliothèque avec plusieurs livres. Chaque livre doit avoir un titre, un auteur, une année de publication, et un prix.
2. Ajoutez un attribut pour indiquer la devise du prix (par exemple : FCFA, USD, etc.).
3. Incluez une balise racine, et assurez-vous que toutes les balises sont correctement fermées et que les attributs sont entourés de guillemets.
4. Validez votre document en vous assurant qu'il respecte les règles suivantes :
 - a. Une seule balise racine.
 - b. Les noms des balises et des attributs sont conformes aux conventions XML.
 - c. Les balises sont bien fermées et les attributs correctement renseignés.

❖ DTD (Document Type Definition)

Les DTD sont des fichiers de définition pour documents XML qui établissent les règles de structure et de contenu des balises. Les DTD peuvent être **internes** (directement incluses dans le document XML) ou **externes** (stockées dans un fichier .dtd).

→ *Syntaxe de définition des éléments avec DTD*

Pour définir les règles associées aux balises, le mot-clé ELEMENT est utilisé.

- **Syntaxe :**

```
<!ELEMENT balise (contenu)>
```

- **Exemple :**

```
<!ELEMENT nom (#PCDATA)>  
<nom>Maurice</nom>
```

Cas spécifiques dans DTD

1. **Définir l'inclusion d'une balise :** Pour inclure une balise à l'intérieur d'une autre, on utilise la syntaxe suivante :

```
<!ELEMENT personne (nom)>
```

Si une balise référencée (ex. <nom>) n'est pas définie, le document ne sera pas valide.

Exemple :

```
<personne>  
  <nom>Charles GBOYOU</nom>  
</personne>
```

2. **Balise contenant une valeur :** On utilise le mot-clé #PCDATA pour indiquer que la balise contient du texte brut.

```
<!ELEMENT personne (nom)>  
<!ELEMENT nom (#PCDATA)>
```

3. **Balise vide :** Le mot-clé EMPTY est utilisé pour indiquer qu'une balise ne contient pas de contenu.

```
<!ELEMENT nom EMPTY>
```

Exemple XML :

```
<personne>  
  <nom />  
</personne>
```

4. **Balise pouvant contenir n'importe quel contenu :** Le mot-clé ANY permet à une balise d'accepter n'importe quel contenu.

```
<!ELEMENT nom ANY>
```

Exemple XML :

```
<personne>  
  <nom>Charles GBOYOU</nom>  
</personne>
```

→ **Structurer le contenu des balises avec DTD**

1. Séquence :

La séquence impose un ordre précis pour les sous-éléments.

a. Syntaxe :

```
<!ELEMENT personne (nom, prenom, age)>
```

b. Exemple XML :

```
<personne>  
  <nom>Dupont</nom>  
  <prenom>Jean</prenom>  
  <age>30</age>  
</personne>
```

2. Liste de choix :

Permet de spécifier que la balise contiendra l'un des éléments énumérés.

a. Syntaxe :

```
<!ELEMENT personne (nom | prenom)>
```

b. Exemple XML :

```
<personne>  
  <nom>Dupont</nom>  
</personne>
```

3. Balise optionnelle :

Une balise optionnelle est marquée par un point d'interrogation ?.

a. Syntaxe :

```
<!ELEMENT personne (nom, prenom?)>
```

b. Exemple XML :

```
<personne>  
  <nom>Dupont</nom>  
</personne>
```

4. Balise répétée optionnelle :

Pour permettre une répétition optionnelle d'une balise, on utilise une étoile *.

a. Syntaxe :

```
<!ELEMENT repertoire (personne*)>
```

b. Exemple XML valide :

```
<repertoire>  
  <personne>  
    <nom>Dupont</nom>  
    <prenom>Jean</prenom>  
  </personne>  
  <personne>  
    <nom>Martin</nom>  
    <prenom>Paul</prenom>  
  </personne>  
</repertoire>
```

5. Balise répétée obligatoire :

Pour indiquer qu'une balise peut se répéter au moins une fois, on utilise un plus +.

a. Syntaxe :

```
<!ELEMENT repertoire (personne+)>
```

b. Exemple XML valide :

```
<repertoire>
  <personne>
    <nom>Dupont</nom>
    <prenom>Jean</prenom>
  </personne>
</repertoire>
```

Un document XML est considéré comme valide lorsqu'il respecte toutes les règles définies dans sa DTD ou son schéma. Ces règles portent sur la structure, les types de données, et l'organisation des balises. Il existe des DTD **internes** (écrites dans le document XML lui-même) et **externes** (fichiers séparés avec l'extension .dtd), offrant de la flexibilité dans la définition et la réutilisation de la structure XML.

En XML, le mot-clé ELEMENT est fondamental pour établir les règles de structure, facilitant ainsi la validation et garantissant que les données XML répondent aux exigences de leur application spécifique.

❖ DTD : Les attributs et les entités

→ *Les attributs dans les DTD*

Pour définir des règles associées aux attributs d'une balise, on utilise le mot-clé ATTLIST. La syntaxe de base est la suivante :

Syntaxe :

```
<!ATTLIST balise attribut type mode>
```

Cette déclaration se compose de cinq éléments : ATTLIST, le nom de la balise, le nom de l'attribut, le type de l'attribut et le mode (obligatoire ou optionnel). Voici quelques cas d'utilisation :

1. **Attribut avec une liste de valeurs possibles :** Dans certains cas, on souhaite restreindre les valeurs d'un attribut à une liste prédéfinie. On utilise pour cela la syntaxe suivante :

```
<!ATTLIST personne sexe (masculin | féminin) mode>
```

Exemple XML :

```
<!-- valide -->
<personne sexe="masculin" />
<!-- valide -->
<personne sexe="féminin" />
<!-- invalide -->
<personne sexe="autre" />
```

2. **Attribut contenant du texte non "parsé" (CDATA) :** Si l'attribut peut contenir tout type de texte sans restriction (texte brut, nombres, lettres), on utilise le mot-clé CDATA :

```
<!ATTLIST personne sexe CDATA mode>
```

Exemple XML :

```
<!-- valide -->
<personne sexe="masculin" />
<personne sexe="12" />
```

3. **Attribut de type identifiant unique (ID) :** Pour qu'un attribut soit unique dans le document, on utilise le type ID :

```
<!ATTLIST personne position ID mode>
```

Exemple XML :

```
<!-- valide -->
<personne position="1" />
<personne position="2" />
<!-- invalide : duplication de l'ID -->
<personne position="1" />
```

4. **Attribut de type référence à un identifiant unique (IDREF) :** Pour référencer un autre ID unique, on utilise IDREF :

```
<!ATTLIST fils pere IDREF mode>
```

Exemple XML :

```
<pere id="PER-1" />
```

```
<file id="PER-2" pere="PER-1" />
```

→ **Modes d'attributs dans les DTD**

Le mode détermine si un attribut est obligatoire, optionnel ou possède une valeur par défaut.

1. **Attribut obligatoire : #REQUIRED**

```
<!ATTLIST personne sexe (masculin | féminin) #REQUIRED>
```

2. **Attribut optionnel : #IMPLIED**

```
<!ATTLIST personne sexe CDATA #IMPLIED>
```

3. **Attribut avec valeur par défaut :**

```
<!ATTLIST personne sexe CDATA "masculin">
```

4. **Attribut avec valeur fixe : #FIXED**

```
<!ATTLIST objet devise CDATA #FIXED "Euro">
```


→ **Les entités dans les DTD**

Les entités permettent de définir des alias pour réutiliser des valeurs ou fragments de texte.

1. **Entités générales** : Ce sont des alias réutilisables dans le document XML.

```
<!ENTITY samsung "Samsung">  
<marque>&samsung;</marque>
```

2. **Entités paramètres** : Utilisées uniquement dans les DTD, elles permettent de référencer des valeurs réutilisables.

```
<!ENTITY % listeMarques "marque (Samsung | Apple) #REQUIRED">
```

3. **Entités externes** : Les entités externes référencent des données stockées dans des fichiers séparés.

```
<!ENTITY samsung SYSTEM "samsung.xml">
```

→ Les DTD internes et externes

Les DTD peuvent être incluses de deux façons : directement dans le document XML (DTD interne) ou dans un fichier séparé (DTD externe).

- **DTD interne** : Définie directement dans le document XML sous le prologue.

```
<?xml version="1.0" encoding="UTF-8"?>
<!DOCTYPE boutique [
  <!ELEMENT boutique (telephone*)>
  <!ELEMENT telephone (marque, modele)>
]>
```

- **DTD externe** : Stockée dans un fichier .dtd séparé, référencée via le mot-clé SYSTEM ou PUBLIC.

```
<!DOCTYPE boutique SYSTEM "boutique.dtd">
```

Ces définitions DTD assurent que le document XML respecte une structure préétablie, garantissant ainsi une compatibilité et une cohérence dans l'utilisation des données.

Exercice 1 : Création d'un document XML simple

Objectif : Apprendre à créer un document XML bien formé.

1. Créez un document XML représentant une **bibliothèque** contenant plusieurs **livres**.
2. Chaque livre doit avoir un **titre**, un **auteur**, une **année de publication**, et un **prix**.
3. Ajoutez des attributs facultatifs pour préciser le **genre** du livre (roman, science-fiction, etc.).
4. Assurez-vous que votre document est bien formé avec une balise racine unique et toutes les balises correctement fermées.

Exercice 2 : Création d'une DTD interne pour un document XML

Objectif : Définir une structure de document XML en utilisant une DTD interne.

1. En utilisant le document XML créé dans l'exercice 1, ajoutez une DTD interne qui impose la présence des éléments suivants :
 - a. Une balise racine bibliotheque.
 - b. Des éléments livre avec des sous-éléments titre, auteur, annee, et prix.
2. Ajoutez un attribut devise pour l'élément prix avec une valeur par défaut "Euro".
3. Validez le document XML en vous assurant qu'il est conforme à la DTD.

Exercice 3 : Création d'une DTD externe

Objectif : Apprendre à créer et à lier une DTD externe.

1. Déplacez la DTD de l'exercice précédent dans un fichier externe nommé bibliotheque.dtd.
2. Modifiez votre document XML pour référencer la DTD externe en utilisant SYSTEM.
3. Ajoutez des éléments supplémentaires à votre document XML pour valider la structure imposée par la DTD externe.

TP :

1 - Créer un document XML pour gérer une **bibliothèque universitaire**. Cette bibliothèque contient plusieurs types de ressources (livres, articles, et supports multimédias) ainsi que des informations sur les auteurs, éditeurs, et catégories de chaque ressource. Vous devez inclure les éléments et attributs suivants dans le document XML.

Instructions :

1. Éléments de la bibliothèque :

- a. La racine du document doit être *bibliotheque*.
- b. La bibliothèque contient plusieurs **ressources**.

2. Ressources :

- a. Chaque ressource doit avoir un type (livre, article, multimedia), défini par un attribut *type*.
- b. Les sous-éléments d'une ressource varient en fonction de son type :
 - i. **livre** : contient les sous-éléments obligatoires *titre*, *auteur*, *annee*, *editeur*, et un attribut optionnel *genre*.
 - ii. **article** : contient les sous-éléments obligatoires *titre*, *auteur*, *publication*, *pages* et un attribut *revue* (obligatoire).
 - iii. **multimedia** : contient les sous-éléments *titre*, *format* et un sous-élément optionnel *description*.

3. Auteurs :

- a. Chaque ressource peut avoir plusieurs auteurs, chacun ayant un identifiant unique *id*.
- b. Les auteurs sont définis séparément dans un élément *auteurs* contenant plusieurs éléments *auteur* (*nom*, *prenom*, *affiliation*).
- c. Les ressources référencent les auteurs par leurs identifiants *IDREF* (ex. *auteur_ref*).

4. Entités :

- a. Définissez une entité pour représenter une devise par défaut (ex. EUR pour euro).
- b. Utilisez cette entité dans les éléments de prix.

5. Exigences supplémentaires :

- a. Les valeurs possibles pour le genre d'un livre doivent être limitées à "roman", "essai", ou "manuel".

- b. Les formats de multimedia doivent être "CD", "DVD", ou "Blu-ray".
- c. Assurez-vous que chaque ressource est correctement structurée selon son type.

2 - Créez une DTD (bibliotheque.dtd) qui valide la structure du document XML en respectant les critères suivants :

1. **Éléments obligatoires et optionnels** : Assurez-vous que chaque type de ressource (livre, article, multimedia) respecte les exigences de sous-éléments obligatoires ou optionnels.
2. **Attributs avec listes de valeurs possibles** : Limitez les valeurs de l'attribut genre à "roman", "essai", ou "manuel", et celles de format pour multimedia à "CD", "DVD", ou "Blu-ray".
3. **Attributs de type ID et IDREF** : Utilisez des attributs ID pour chaque auteur, et référencez-les dans les ressources avec IDREF.
4. **Entités** : Définissez une entité pour la devise par défaut (EUR) et utilisez-la dans le prix des livres.

Schéma XML

❖ Limitations des DTD

Les DTD (Définitions de Type de Document) présentent certaines limites : elles ne sont pas écrites en XML et ne permettent pas la gestion des typages de données avancés. Pour pallier ces limitations, les schémas XML sont souvent préférés.

❖ Caractéristiques du Schéma XML

Les fichiers de schéma XML utilisent l'extension .xsd. Comme tout document XML, un schéma XML comporte un prologue et un corps. L'élément racine du corps est toujours `<xsd:schema />`.

Exemple :

```
<!-- Prologue -->
<?xml version="1.0" encoding="UTF-8" ?>
<!-- Élément racine -->
<xsd:schema xmlns:xsd="http://www.w3.org/2001/XMLSchema">
```

❖ Référencement d'un Schéma XML

Pour référencer un schéma XML dans un document XML, on utilise des attributs au niveau de l'élément racine.

- **Schéma avec espace de noms :**

```
xsi:schemaLocation="chemin_vers_fichier.xsd"
```

- **Schéma sans espace de noms :**

```
xsi:noNamespaceSchemaLocation="chemin_vers_fichier.xsd"
```

Exemple complet d'un document XML :

```
<?xml version="1.0" encoding="UTF-8"?>
<racine xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
      xsi:noNamespaceSchemaLocation="chemin_vers_fichier.xsd">
</racine>
```

❖ Schéma XML : Les éléments simples

Un élément simple contient une valeur de type simple sans attributs ni éléments imbriqués. Un attribut peut également être considéré comme un élément simple, car il ne contient qu'une seule valeur de type simple.

Exemple d'éléments simples :

```
<nom>Roger</nom> <!-- Élément simple sans attribut ni élément imbriqué -->
<personne sexe="masculin">Roger DURAND</personne> <!-- Attribut "sexe" de type simple -->
<personne>
  <nom>DURAND</nom> <!-- Élément simple -->
  <prenom>Roger</prenom> <!-- Élément simple -->
</personne>
```

❖ Déclaration d'un élément simple dans un Schéma XML

Pour déclarer un élément simple, on utilise `<xsd:element>` avec les attributs `name` et `type`.

Exemple :

```
<xsd:element name="nom" type="xsd:string" />
<xsd:element name="prenom" type="xsd:string" />
<xsd:element name="age" type="xsd:int" />
```

→ Valeurs par défaut et valeurs fixes

- **Valeur par défaut :** Utilisez l'attribut `default`.

```
<xsd:element name="prenom" type="xsd:string" default="Roger" />
```

- **Valeur constante :** Utilisez l'attribut `fixed`.

```
<xsd:element name="prenom" type="xsd:string" fixed="Robert" />
```

Exemple XML avec valeur par défaut et valeur constante :

```
<prenom>Jean</prenom> <!-- valide -->
<prenom /> <!-- prend la valeur "Roger" par défaut -->
<prenom>Robert</prenom> <!-- valeur constante : valide uniquement si égal à "Robert" -->
```


❖ Déclaration des attributs dans un Schéma XML

Les attributs,

considérés comme des éléments simples, sont déclarés avec `<xsd:attribute>`.

Exemple :

```
<xsd:attribute name="sexe" type="xsd:string" />
```

→ *Valeurs par défaut, obligatoire et constantes pour les attributs*

- **Valeur par défaut** : Utilisez `default`.

```
<xsd:attribute name="prenom" type="xsd:string" default="Charles" />
```

- **Attribut obligatoire** : Utilisez `use="required"`.

```
<xsd:attribute name="prenom" type="xsd:string" use="required" />
```

- **Valeur constante** : Utilisez `fixed`.

```
<xsd:attribute name="prenom" type="xsd:string" fixed="Charles" />
```

❖ Types Simples dans les Schémas XML

Les types simples se divisent en plusieurs catégories, notamment les chaînes de caractères, les dates, les nombres, et les types booléens.

Types chaînes de caractères

Type	Description
string	Chaîne de caractères
normalizedString	Chaîne de caractères normalisée
token	Chaîne de caractères sans espaces initiaux ou finaux
language	Code de langue
Name	Nom XML valide
NCName	Nom XML sans le caractère :

Types dates

Type	Description
date	Représente une date
time	Représente une heure
dateTime	Représente une date et un temps
gYear	Représente une année
duration	Représente une durée

Types numériques

Type	Description
integer	Nombre entier
float	Nombre flottant (32 bits)
double	Nombre flottant (64 bits)
decimal	Nombre décimal

Exemple d'un élément numérique :

```
<xsd:element name="nombre" type="xsd:float" />
```

Types booléens et autres

Type	Description
boolean	Représente vrai ou faux
anyURI	Représente une URI
base64Binary	Donnée binaire en Base64
hexBinary	Donnée binaire en hexadécimal

Schéma XML : Les Éléments Complexes

Un élément complexe peut contenir d'autres éléments et/ou des attributs. Dans un schéma XML, les éléments complexes sont déclarés à l'aide de `<xsd:complexType>`.

Exemple de déclaration d'un élément complexe :

```
<xsd:element name="personne">
  <xsd:complexType>
    <xsd:sequence>
      <xsd:element name="nom" type="xsd:string"/>
      <xsd:element name="prenom" type="xsd:string"/>
    </xsd:sequence>
    <xsd:attribute name="sexe" type="xsd:string"/>
  </xsd:complexType>
</xsd:element>
```

Dans cet exemple, l'élément `personne` contient deux sous-éléments (`nom` et `prenom`) et un attribut (`sexe`)

Types de Contenu Complexe

1. **Contenu Élément** : Utilise `<xsd:sequence>`, `<xsd:choice>`, ou `<xsd:all>` pour structurer l'ordre des sous-éléments :
 - a. **`<xsd:sequence>`** : Les éléments doivent apparaître dans un ordre précis.
 - b. **`<xsd:choice>`** : Permet de choisir entre plusieurs éléments, un seul étant autorisé.
 - c. **`<xsd:all>`** : Tous les éléments peuvent apparaître dans n'importe quel ordre, mais chaque élément peut apparaître au maximum une fois.
2. **Contenu Mixte** : Permet de mélanger texte et sous-éléments, idéal pour des éléments textuels avec des balises formatées.
3. **Contenu Vide** : Utilisé pour les éléments qui n'ont ni texte ni sous-éléments mais peuvent contenir des attributs.

Exemple d'élément avec choix d'éléments :

```
<xsd:element name="contact">
  <xsd:complexType>
    <xsd:choice>
      <xsd:element name="email" type="xsd:string"/>
      <xsd:element name="telephone" type="xsd:string"/>
    </xsd:choice>
  </xsd:complexType>
</xsd:element>
```

Dans cet exemple, contact peut être soit un email soit un telephone, mais pas les deux à la fois.

❖ Types de Restriction dans un Schéma XML

Les schémas XML permettent de restreindre les valeurs d'un élément ou d'un attribut en utilisant des types simples avec des restrictions spécifiques. On utilise pour cela `<xsd:restriction>`.

Exemples de Restrictions :

- **Valeurs autorisées** : Utilise <xsd:enumeration> pour définir une liste de valeurs possibles.

```
<xsd:element name="couleur">
  <xsd:simpleType>
    <xsd:restriction base="xsd:string">
      <xsd:enumeration value="rouge"/>
      <xsd:enumeration value="bleu"/>
      <xsd:enumeration value="vert"/>
    </xsd:restriction>
  </xsd:simpleType>
</xsd:element>
```

- **Plage de valeurs** : Utilise <xsd:minInclusive>, <xsd:maxInclusive>, <xsd:minExclusive>, ou <xsd:maxExclusive> pour limiter des valeurs numériques.

```
<xsd:element name="age">
  <xsd:simpleType>
    <xsd:restriction base="xsd:int">
      <xsd:minInclusive value="0"/>
      <xsd:maxInclusive value="120"/>
    </xsd:restriction>
  </xsd:simpleType>
</xsd:element>
```

- **Longueur de texte** : Utilise `<xsd:length>`, `<xsd:minLength>`, et `<xsd:maxLength>` pour restreindre la longueur des chaînes de caractères.

```
<xsd:element name="codePostal">
  <xsd:simpleType>
    <xsd:restriction base="xsd:string">
      <xsd:length value="5"/>
    </xsd:restriction>
  </xsd:simpleType>
</xsd:element>
```

❖ Utilisation des Groupes dans les Schémas XML

Les groupes d'éléments et d'attributs facilitent la réutilisation des structures XML dans les schémas.

1. **Groupe d'Éléments** : Défini avec `<xsd:group>`, il permet de regrouper des éléments qui apparaîtront ensemble.

```
<xsd:group name="nomCompleet">
  <xsd:sequence>
    <xsd:element name="nom" type="xsd:string"/>
    <xsd:element name="prenom" type="xsd:string"/>
  </xsd:sequence>
</xsd:group>
```

```
<xsd:element name="employe">
  <xsd:complexType>
    <xsd:group ref="nomComplet"/>
    <xsd:element name="poste" type="xsd:string"/>
  </xsd:complexType>
</xsd:element>
```

2. **Groupe d'Attributs** : Défini avec <xsd:attributeGroup>, il permet de regrouper des attributs.

```
<xsd:attributeGroup name="identite">
  <xsd:attribute name="id" type="xsd:int" use="required"/>
  <xsd:attribute name="sexe" type="xsd:string"/>
</xsd:attributeGroup>

<xsd:element name="client">
  <xsd:complexType>
    <xsd:attributeGroup ref="identite"/>
  </xsd:complexType>
</xsd:element>
```


Schéma XML : Définition et Utilisation des Types Complexes

❖ *Définition d'un Élément Complexe*

Un élément complexe est un élément XML qui peut contenir d'autres éléments imbriqués ou inclure des attributs.

Exemples :

```
<!-- Un élément complexe contenant d'autres éléments -->
<personne>
  <nom>GBOYOU</nom>
  <prenom>Charles</prenom>
</personne>

<!-- Un élément complexe avec attribut -->
<personne sexe="masculin">Charles GBOYOU</personne>
```

❖ *Déclaration d'un Élément Complexe*

Pour définir un élément complexe dans un schéma XML, utilisez l'élément `<xsd:complexType>`.

Syntaxe :

```
<xsd:element name="mon_nom">
  <xsd:complexType>
    <!-- contenu ici -->
  </xsd:complexType>
</xsd:element>
```

Exemple :

```
<xsd:element name="personne">
  <xsd:complexType>
    <xsd:sequence>
      <xsd:element name="nom" type="xsd:string"/>
      <xsd:element name="prenom" type="xsd:string"/>
    </xsd:sequence>
    <xsd:attribute name="sexe" type="xsd:string"/>
  </xsd:complexType>
</xsd:element>
```

❖ Types de Contenus Complexes

1. Contenus Simples

- a. Ce type de contenu contient uniquement des attributs et du texte simple.

Exemple :

```
<prix devise="euros">35</prix>
```

Déclaration :

```
<xsd:element name="prix">
  <xsd:complexType>
    <xsd:simpleContent>
      <xsd:extension base="xsd:positiveInteger">
        <xsd:attribute name="devise" type="xsd:string"/>
      </xsd:extension>
    </xsd:simpleContent>
  </xsd:complexType>
</xsd:element>
```

2. Contenus Standards

- a. Ce type de contenu est composé d'éléments enfants (simples ou complexes) ou d'attributs uniquement.

Exemple :

```
<personne>
  <nom>GBOYOU</nom>
  <prenom>Charles</prenom>
</personne>
```

Déclaration :

```
<xsd:element name="personne">
  <xsd:complexType>
    <xsd:sequence>
      <xsd:element name="nom" type="xsd:string"/>
      <xsd:element name="prenom" type="xsd:string"/>
    </xsd:sequence>
  </xsd:complexType>
</xsd:element>
```

```
</xsd:sequence>
</xsd:complexType>
</xsd:element>
```

3. Contenus Mixtes

- a. Un contenu mixte contient des attributs, des éléments et du texte.

Exemple :

```
<facture>
  <acheteur>Rony</acheteur>, doit payer
  <somme>1000</somme>€
</facture>
```

Déclaration :

```
<xsd:element name="facture">
  <xsd:complexType mixed="true">
    <xsd:sequence>
      <xsd:element name="acheteur" type="xsd:string"/>
      <xsd:element name="somme" type="xsd:int"/>
    </xsd:sequence>
  </xsd:complexType>
</xsd:element>
```

Définir la Séquence et le Choix d'Éléments

- **La Séquence** : Définit un ordre strict pour l'apparition des éléments dans l'élément complexe.

Syntaxe :

```
<xsd:element name="mon_nom">
  <xsd:complexType>
    <xsd:sequence>
      <!-- éléments ici -->
    </xsd:sequence>
  </xsd:complexType>
</xsd:element>
```

- **Le Choix** : Permet à un élément de prendre la valeur de l'un des éléments prédéfinis dans la liste.

Syntaxe :

```
<xsd:element name="mon_nom">
  <xsd:complexType>
    <xsd:choice>
      <!-- éléments ici -->
    </xsd:choice>
  </xsd:complexType>
</xsd:element>
```

❖ Gestion du Nombre d'Occurrences

Par défaut, un élément n'apparaît qu'une seule fois. Utilisez minOccurs pour le nombre minimal d'occurrences et maxOccurs pour le maximum. Pour un nombre illimité, utilisez maxOccurs="unbounded".

Exemple :

```
<xsd:complexType name="personne">
  <xsd:sequence>
    <xsd:element name="nom" type="xsd:string"/>
    <xsd:element name="prenom" type="xsd:string" minOccurs="2" maxOccurs="unbounded"/>
  </xsd:sequence>
</xsd:complexType>
```

❖ Réutilisation de Types Complexes

Pour réutiliser des structures complexes, on utilise le mot clé ref.

```
<xsd:element ref="nom_ref"/>
```

Exemple :

```
<xsd:element name="Montant" type="xsd:string"/>
<xsd:element name="compte">
  <xsd:complexType>
    <xsd:sequence>
      <xsd:element ref="Montant"/>
    </xsd:sequence>
  </xsd:complexType>
</xsd:element>
```

Exercices

Exercice 1 : Définir un Élément Complexe Simple : Créez un schéma XML pour le document suivant, qui représente un produit avec un nom et un prix, où le prix doit inclure une devise comme attribut.

```
<produit>
  <nom>Stylo</nom>
  <prix devise="USD">1.99</prix>
</produit>
```

Écrivez le schéma XML qui déclare cet élément complexe, en définissant nom comme une chaîne (string) et prix comme un nombre (decimal).

Exercice 2 : Utiliser la Séquence et les Occurrences: Dans cet exercice, nous avons une liste de commandes dans laquelle chaque commande contient un ou plusieurs articles, identifiés par un nom et une quantité.

```
<commande>
  <article>
    <nom>Clavier</nom>
    <quantite>2</quantite>
  </article>
  <article>
    <nom>Souris</nom>
    <quantite>1</quantite>
  </article>
</commande>
```

Définissez le schéma XML pour cet élément complexe en utilisant une séquence et spécifiez que l'élément article peut apparaître plusieurs fois (minOccurs=1, maxOccurs="unbounded").

Exercice 3 : Utiliser le Choix et le Contenu Simple

Vous devez créer un schéma XML pour un élément contact qui contient soit un numéro de téléphone, soit une adresse email.

Document XML :

```
<contact>  
  <email>contact@example.com</email>  
</contact>
```

ou

```
<contact>  
  <telephone>+123456789</telephone>  
</contact>
```

Déclarez le schéma XML pour l'élément contact, en utilisant <choice> pour permettre soit un élément email (de type string), soit un élément telephone (de type string).

Exercice :

L'exercice suivant propose un document XML complexe, représentant un client d'une entreprise, avec ses informations personnelles, son adresse, ses contacts, et une liste de commandes passées.

Document XML :

```
<client>
  <identite>
    <nom>Dupont</nom>
    <prenom>Marie</prenom>
  </identite>
  <adresse type="residence">
    <ligne1>123 Rue de la Paix</ligne1>
    <ville>Paris</ville>
    <codePostal>75001</codePostal>
    <pays>France</pays>
  </adresse>
  <contacts>
    <email>marie.dupont@example.com</email>
    <telephone>+33123456789</telephone>
  </contacts>
  <commandes>
    <commande date="2024-11-01">
      <article>
        <nom>Stylo</nom>
        <quantite>2</quantite>
        <prix devise="EUR">1.99</prix>
      </article>
      <article>
```

```
<nom>Cahier</nom>
<quantite>5</quantite>
<prix devise="EUR">2.50</prix>
</article>
</commande>
<commande date="2024-10-15">
  <article>
    <nom>Classeur</nom>
    <quantite>3</quantite>
    <prix devise="EUR">4.75</prix>
  </article>
</commande>
</commandes>
</client>
```

Question :

Écrivez le schéma XML pour ce document. Prenez en compte les éléments suivants :

1. Utilisez <sequence> pour définir l'ordre des éléments dans les sections identité, adresse, et commandes.
2. Déclarez des attributs pour adresse (type) et prix (devise).
3. Utilisez minOccurs et maxOccurs pour gérer les occurrences de l'élément article dans chaque commande.
4. Assurez-vous que l'élément commande contient la date de commande en attribut (date).
5. Utilisez des types complexes imbriqués pour définir les différentes sections du document (identité, adresse, contacts, et commandes).

DOM (Document Object Model)

❖ Définition :

Le DOM (Document Object Model) est une représentation objet d'un document XML. Il permet :

- **Lire** un document XML pour en extraire des informations (éléments, attributs, commentaires, etc.) afin de les exploiter.
- **Écrire** un document XML.

❖ Notion d'arbre :

Un document XML peut être représenté sous forme d'arbre.

Exemple :

```
<?xml version="1.0" encoding="UTF-8" standalone="yes" ?>
<repertoire>
  <personne sexe="masculin">
    <nom>GBOYOU</nom>
    <prenom>Charles</prenom>
    <telephones>
      <telephone type="fixe">02 03 04 05</telephone>
      <telephone type="portable">07 08 09 10</telephone>
    </telephones>
    <adresses>
      <pays>Benin</pays>
      <rue>12</rue>
    </adresses>
  </personne>
</repertoire>
```

Types de nœuds

1. **Document** :

- Représente l'ensemble du document.
- Contient une référence à l'élément racine (de type `Element`).
- Permet de construire différents types de nœuds.

2. **DocumentType** : Associé à une DTD, caractérise les entités internes/externes (sauf les entités paramétriques).

3. **EntityReference** : Représente une référence d'entité insérée dans un texte.

4. **Element** :

- Le type de nœud le plus utilisé.
- Représente une balise avec des méthodes pour accéder aux nœuds fils, parents, et attributs.

5. **Attr** : Représente un attribut avec un nom et une valeur.

6. **Comment** : Représente un commentaire. Sa gestion dépend des options du parseur.

7. **Text** : Représente un nœud texte. Les nœuds texte adjacents peuvent être normalisés.

❖ Opérations sur l'arbre DOM

→ Opérations de lecture :

Voici quelques méthodes courantes de l'interface `Node` :

- **getAttributes()**: Retourne les attributs d'un élément sous forme de `NamedNodeMap`.
- **getChildNodes()**: Liste des nœuds fils.
- **getFirstChild()** / **getLastChild()**: Premier/dernier nœud fils.
- **getNextSibling()** / **getPreviousSibling()**: Nœud adjacent (avant ou après).
- **getParentNode()**: Nœud parent.
- **getOwnerDocument()**: Document propriétaire du nœud.
- **getTextContent()**: Concaténation du contenu texte des nœuds.

→ **Opérations de modification :**

Voici quelques méthodes de l'interface Node :

- `appendChild(Node newChild)`: Ajout d'un fils.
- `removeChild(Node oldChild)`: Suppression d'un fils.
- `replaceChild(Node newChild, Node oldChild)`: Remplacement d'un fils.
- `cloneNode(boolean deep)`: Retourne un clone du nœud.
- `normalize()`: Élimine les textes adjacents pour homogénéiser l'arbre.

→ **Interfaces importantes**

▪ **Document :**

- `getDocumentElement()`: Retourne l'élément racine.
- `getElementById(String elementId)`: Retourne un élément par son ID.
- `getElementsByTagName(String tagName)`: Retourne tous les éléments d'un certain nom.
- `createElement(String tagName)`: Crée un nouvel élément.
- `createTextNode(String data)`: Crée un nœud texte.
- `createComment(String data)`: Crée un commentaire.

▪ **Element :**

- `getAttribute(String name) / setAttribute(String name)`: Lecture/écriture d'un attribut.
- `getTagName()`: Retourne le nom de l'élément.
- `removeAttribute(String name)`: Supprime un attribut.

▪ **Attr :**

- getName(): Nom de l'attribut.
- getValue() / setValue(String value): Lecture/modification de la valeur.
- getOwnerElement(): Élément associé.
- isId(): Indique si l'attribut est de type ID.

▪ **Text :**

- Hérite de CharacterData.
- Méthodes courantes :
 - appendData(String arg): Ajoute un texte.
 - deleteData(int offset, int count): Supprime un texte à une position donnée.
 - insertData(int offset, String arg): Insère un texte à une position donnée.
 - replaceData(int offset, int count, String arg): Remplace une portion de texte.

Exercice : Manipulation du DOM avec un document XML

1. Chargez le document XML ci-dessus dans un programme en utilisant une bibliothèque de manipulation DOM de votre choix (exemple : Java, Python, ou JavaScript).
2. Réalisez les opérations suivantes :
 - Affichez le nom complet (nom + prénom) de la personne.
 - Modifiez le numéro de téléphone **fixe** en "01 23 45 67".
 - Ajoutez un nouveau numéro de téléphone **portable** avec la valeur "09 87 65 43".
 - Supprimez l'attribut sexe de l'élément personne.
 - Ajoutez un nouvel élément <ville> sous <adresses> avec la valeur "Cotonou".
3. Affichez le document XML après toutes ces modifications.

Correction :

Solution en Python avec la bibliothèque : **xml.dom.minidom** :

```
from xml.dom.minidom import parse
import os

# Charger le fichier XML
base_dir = os.path.dirname(os.path.abspath(__file__))
xml_file_path = os.path.join(base_dir, "repertoire.xml")

if not os.path.exists(xml_file_path):
    print(f"Erreur : Le fichier '{xml_file_path}' n'existe pas.")
    exit(1)

doc = parse(xml_file_path) # Lire et parser le fichier XML

# 1. Afficher le nom complet
nom = doc.getElementsByTagName("nom")[0].firstChild.data
prenom = doc.getElementsByTagName("prenom")[0].firstChild.data
print("Nom complet :", nom, prenom)

# 2. Modifier le téléphone fixe
telephone_fixe = doc.getElementsByTagName("telephone")[0]
telephone_fixe.firstChild.data = "01 23 45 67"
```

3. Ajouter un nouveau téléphone portable

```
telephones = doc.getElementsByTagName("telephones")[0]
new_phone = doc.createElement("telephone")
new_phone.setAttribute("type", "portable")
new_phone.appendChild(doc.createTextNode("09 87 65 43"))
telephones.appendChild(new_phone)
```

4. Supprimer l'attribut `sexe`

```
personne = doc.getElementsByTagName("personne")[0]
personne.removeAttribute("sexe")
```

5. Ajouter une nouvelle ville

```
adresses = doc.getElementsByTagName("adresses")[0]
ville = doc.createElement("ville")
ville.appendChild(doc.createTextNode("Cotonou"))
adresses.appendChild(ville)
```

Afficher le document XML modifié

```
print("\nDocument XML modifié :")
print(doc.toprettyxml(indent="  "))
```

Sauvegarder les modifications dans un nouveau fichier XML

```
with open("repertoire_modifie.xml", "w", encoding="utf-8") as f:
    f.write(doc.toprettyxml(indent="  "))
    print("\nModifications sauvegardées dans 'repertoire_modifie.xml'")
```


XPath : Utilisation, Axes, Tests de Nœuds et Prédicats

```
<?xml version="1.0" encoding="UTF-8"?>
<repertoire>
  <personne sexe="masculin">
    <nom>GBOYOU</nom>
    <prenom>Charles</prenom>
    <telephones>
      <telephone type="fixe">02 03 04 05</telephone>
      <telephone type="portable">07 08 09 10</telephone>
    </telephones>
    <adresses>
      <pays>Benin</pays>
      <rue>12</rue>
    </adresses>
  </personne>
  <personne sexe="masculin">
    <nom>AVOCETIEN</nom>
    <prenom>Jean</prenom>
    <telephones>
      <telephone type="fixe">01 23 45 67</telephone>
      <telephone type="portable">09 87 65 43</telephone>
    </telephones>
    <adresses>
      <pays>France</pays>
      <rue>34</rue>
    </adresses>
  </personne>
</repertoire>
```

❖ Qu'est-ce que XPath ?

XPath est un langage utilisé pour naviguer dans les documents XML. Il permet de localiser des nœuds spécifiques ou de filtrer des ensembles de nœuds en fonction de divers critères.

❖ Tableau récapitulatif des axes

Nom de l'axe	Description
ancestor	Orienté la recherche vers les ancêtres du nœud courant.
ancestor-or-self	Orienté la recherche vers le nœud courant et ses ancêtres.
attribute	Orienté la recherche vers les attributs du nœud courant.
child	Orienté la recherche vers les enfants du nœud courant.
descendant	Orienté la recherche vers les descendants du nœud courant.
descendant-or-self	Orienté la recherche vers le nœud courant et ses descendants.
following	Orienté la recherche vers les frères suivants du nœud courant.
parent	Orienté la recherche vers le parent du nœud courant.
preceding	Orienté la recherche vers les nœuds précédant le nœud courant.
preceding-sibling	Orienté la recherche vers les frères précédents du nœud courant.
self	Orienté la recherche vers le nœud courant.

Exercice pratique : Trouver tous les enfants du nœud repertoire.

```
/child::repertoire/child::*
```

❖ **Tableau récapitulatif des tests des nœuds**

Nom du test de nœud	Description
nom du nœud	Orienté la recherche vers un nœud dont le nom a été spécifié explicitement.
*	Orienté la recherche vers tous les nœuds, indépendamment de leur nom.
node()	Orienté la recherche vers tous les types de nœuds (éléments, commentaires, etc.).
text()	Orienté la recherche vers les nœuds de type texte.
comment()	Orienté la recherche vers les nœuds de type commentaire.

Exercice pratique : Trouver tous les nœuds texte sous adresses.

```
/repertoire/personne/adresses/child::text()
```

4. Exemple complet avec axes

Récupérer le pays de domiciliation de GBOYOU Charles.

Étapes :

1. Descendre dans le nœud repertoire.
2. Descendre dans le nœud personne.
3. Descendre dans le nœud adresses.
4. Descendre dans le nœud pays.

Expression XPath correspondante :

```
/child::repertoire/child::personne/child::adresses/child::pays
```

❖ Tableau récapitulatif des prédicats

Nom du prédicat	Description
attribute	Affine la recherche en fonction d'un attribut.
count()	Compte le nombre de nœuds correspondant au chemin spécifié.
last()	Sélectionne le dernier nœud d'une liste.
position()	Affine la recherche en fonction de la position d'un nœud dans une liste.

Exemple pratique avec prédicat :

Récupérer le nœud contenant le numéro de téléphone fixe de GBOYOU Charles.

```
/repertoire/personne/telephones/telephone[attribute::type="fixe"]
```

Exercice pratique - Niveau Avancé

Récupérer le dernier téléphone portable de GBOYOU Charles.

```
/repertoire/personne/telephones/telephone[@type="portable"][last()]
```

Exercice pratique supplémentaire

Trouver tous les attributs du nœud personne.

```
/repertoire/personne/attribute::*
```

Compter le nombre total de téléphones dans le répertoire.

```
count(/repertoire/personne/telephones/telephone)
```

Trouver le texte du premier téléphone fixe.

```
/repertoire/personne/telephones/telephone[@type="fixe"][position()=1]/text()
```

Récupérer tous les noms des personnes :

```
/repertoire/personne/nom
```

Récupérer le pays de domiciliation de AVOCETIEN Jean :

```
/repertoire/personne[nom="AVOCETIEN" and prenom="Jean"]/adresses/pays
```

Compter le nombre de téléphones portables :

```
count(/repertoire/personne/telephones/telephone[@type="portable"])
```

Récupérer les numéros de téléphone fixe :

```
/repertoire/personne/telephones/telephone[@type="fixe"]
```

Trouver les prénoms de toutes les personnes dans le répertoire :

```
/repertoire/personne/prenom
```

Récupérer les noms des personnes vivant en France :

```
/repertoire/personne[adresses/pays="France"]/nom
```

Récupérer la rue de la dernière personne du répertoire :

```
/repertoire/personne[last()]/adresses/rue
```

Fin